

TECHNICAL AUDIT REPORT

PJChatbot v2 - Architecture & API Optimization

Target System:	PJChatbot Administration and Client Dashboard v2.0
Developer Team:	Hospital Information System (HIS) Integration Unit
Auditor Agent:	Antigravity AI Pair Programming Agent
Date of Audit:	August 20, 2026
Document Status:	Completed Audit & Patched

Thammasat University Hospital (TUH) - Proprietary & Confidential

1. System Architecture Overview

The PJChatbot v2 application is built on a split-stack modern architecture. The backend serves as a high-performance RESTful API using FastAPI (Python), while the frontends (administration panel and public chatbot interface) are built on top of Vite and React.js.

Backend Stack Components:

- Core API Framework: FastAPI (Python 3.12/3.14) with asynchronous operations (AsyncIO).
- Database ORM Layer: SQLAlchemy 2.0 with async SQLite driver (aiosqlite).
- Vector Store: Chroma DB (local_chroma) for dense vector search and document embedding storage.
- Term Indexing: PyThaiNLP combined with BM25 algorithm for exact keyword matching.
- NLP Embeddings: BAAI/bge-m3 text embedding model loaded locally into GPU/CPU memory.

Frontend Stack Components:

- Build Tool: Vite for hot-module reloading and lightning-fast developer experience.
- UI Library: React.js (v18) with states managed via React hooks (useState, useEffect, useRef).
- Design/Styling: Vanilla CSS with custom scrollbars combined with Tailwind CSS classes for responsive grid layouts.

Hybrid Retrieval Engine (RAG Pipeline):

The chatbot uses a state-of-the-art Hybrid Retriever. Upon receiving a query:

1. Dense Retrieval: Queries Chroma DB for semantic vector matching (using BGE-M3 embeddings).
2. Sparse Retrieval: Queries the BM25 index for exact phrase and Thai term matching.
3. Fusion: Normalizes and blends retrieval scores using Reciprocal Rank Fusion (RRF).
4. LLM Generation: Sends top retrieved chunks to a local Ollama instance or cloud Gemini API to generate the response.

2. Code Audit & Alignment Review

During this iteration, several severe inconsistencies between the React frontends and the FastAPI backend were discovered. These led to fatal errors where administrators were unable to upload, delete, or modify welfare forms, documents, and announcements.

Case Study A: Announcements Management

The React admin panel expected RPC-style POST actions to manage announcements (e.g. `/api/admin/announcements/create``, `/api/admin/announcements/delete``). The FastAPI backend, however, only implemented RESTful APIs (e.g. `POST /announcements``, `DELETE /announcements/{id}``). This resulted in HTTP 405 Method Not Allowed exceptions when the frontend attempted to create or delete announcements.

Case Study B: Feedback Logging Duplication

A critical bug was flagged where clicking the 'Dislike' (Thumbs Down) button once created up to 3 separate records in the SQL database. The audit revealed that this was caused by an anti-pattern in the React code:

1. The `fetch()` request to submit feedback was called inside the callback of the `setSessions()` state updater function.
2. React's Strict Mode executes state updater functions twice per render to detect side-effects. This caused two duplicate database inserts with empty comments.
3. When the user entered a comment in the modal and hit submit, a third request was sent, creating a third record.

Case Study C: PDF Page Anchors

PDF page navigation was originally built using query parameters (e.g. `?page=3``). In standard web browsers, query parameters do not trigger native PDF readers to scroll to a specific page. Standard PDF document fragments must use hashes (e.g. `#page=3``).

3. Implemented Solutions & Optimization

All issues flagged in the code audit have been resolved and successfully verified:

1. Legacy RPC Endpoint Wrappers (Backend)

To maintain 100% backwards compatibility and avoid changing the frontend structure, compatibility endpoints were added to ``v2/backend/app/routers/admin.py`` for document processing, welfare forms, and announcements. For example, POST wrappers for deletion and creation were mapped to direct ORM commands:

```
@router.post("/announcements/delete")
async def delete_announcement_compatibility(payload: LegacyAnnouncementDeleteRequest, ...):
    result = await db.execute(select(Announcement).where(Announcement.id == payload.id))
    ann = result.scalar_one_or_none()
    if ann:
        await db.delete(ann)
        await db.commit()
    return {"success": True}
```

2. Feedback Deduplication & Upsert Logic (Backend & Frontend)

- Frontend: Moved the feedback ``fetch`` call out of the React state updater callback, preventing Strict Mode double-firing.
- Backend: Implemented upsert logic in ``/api/admin/feedback/submit``. If a feedback entry already exists with the same query and answer, it updates the rating and comment of the existing row instead of appending duplicate records.

3. Browser-Native Page Jumping (Frontend & Backend)

Changed all page link generators in both the chatbot citations and the admin log panel to use the ``#page=N`` fragment. Clicking the Chunk ID or citation link now opens the PDF in Chrome/Firefox and scrolls instantly to the correct page.

4. Refactoring Recommendations & Component Model

To improve code maintainability and scalability, we propose the following component architecture updates:

1. Modularize App.jsx into Component Subfolders

Currently, `v2/admin/src/App.jsx` stands at ~4,900 lines of code. This should be broken down into clean folders:

- `src/components/common/`: Sidebar, Header, ModalContainer.
- `src/components/dashboard/`: StatsCard, PortalCard.
- `src/components/tabs/`: TabDocuments, TabForms, TabAnnouncements, TabHistory.

By utilizing prop drilling or React Context (e.g. AdminContext) to pass shared states, the main `App.jsx` file size can be reduced to less than 300 lines.

2. Standardize API Caching with React Query / Axios

Instead of multiple isolated `fetch` calls scattered inside tabs, we recommend introducing React Query (`@tanstack/react-query`). This offers automatic caching, background refetching, and eliminates loading state variables (`loadingHistory`, `loadingUsers`, etc.).

3. Implement Request Validation Middleware

The React frontend should implement axios interceptors or a unified `apiClient` wrapper to check response statuses. Currently, if the backend returns a 405 error page or a 500 JSON, the frontend might display 'Success' because it only checks if `data.error` is present. An HTTP interceptor would catch status codes outside the 2xx range and throw standard errors.